Bilkent University

Electrical and Electronics Engineering Department

Lab 2 Report: Introduction to VHDL

Purpose

The main goal of this experiment is to learn how to design, debug, and implement combinational logic circuits using VHDL and the Vivado Design Suite. The process focuses on analyzing a faulty VHDL code to identify and fix errors, which highlights the importance of debugging in digital design. Additionally, the experiment aims to verify that the circuit works correctly by running a testbench simulation and physically implementing the design on a Basys3 FPGA board.

Design Specifications

This project was developed using Xilinx Vivado and targeted for the Basys3 FPGA board, which is based on the Artix-7 architecture. The system is a combinational circuit with a hierarchical structure, consisting of a top_module and two sub-modules (sub_module1 and sub_module2). It takes an 8-bit input from the physical switches (i_SW) and displays the result as an 8-bit output on the LEDs (o_LED).

Specific modifications were made to sub_module1 based on the student ID requirements. For this design, the logic gates in lines 14-16 were updated to **OR, OR, and XOR** operations. In the top-level architecture, the output of the second submodule is converted to an unsigned integer, added to a constant value of 25, and then XORed with the output of the first submodule to generate the final signals for the LEDs.

Methodology

"In VHDL, the inputs and outputs of a module are defined in the **Entity** section. This part of the code specifies the port names, directions (in, out, or inout), and data types like STD_LOGIC or STD_LOGIC_VECTOR. For example, the top_module in this experiment declares an 8-bit input vector (i_SW) and an 8-bit output vector (o_LED). Additionally, VHDL allows a module to be used inside another by declaring it as a **component** within the architecture."

Task 1: Modification of Logic Gates

In the first task, I modified the sub_module1.vhd file to implement specific logic operations. According to the lab requirements, the logic gates were selected based on the last digit of my student ID number (which is 7). Consequently, I updated lines 14, 15, and 16 of the code to use **OR**, **OR**, and **XOR** gates, respectively. The modified code

snippet is shown in **Figure 1**.1.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity sub_module1 is
Port (
i_input_byte  : in  STD_LOGIC_VECTOR (7 downto 0);
o_output_byte : out STD_LOGIC_VECTOR (7 downto 0)
);
end sub_module1;

architecture Structural_sub1 of sub_module1 is

begin
o_output_byte(0)          <= i_input_byte(0) or i_input_byte(1); --Change these th
o_output_byte(1)          <= i_input_byte(2) or i_input_byte(3);
o_output_byte(2)          <= i_input_byte(4) xor i_input_byte(5);
o_output_byte(5 downto 3) <= "010";
o_output_byte(7 downto 6) <= (others => '0');

end Structural_sub1;
```

Figure 1.1 Modified Gate Codes

Operations	Last digit of ID number
AND OR XOR	0
XNOR NAND OR	1
XOR XNOR AND	2
AND XOR XNOR	3
OR AND NAND	4
NAND XOR XNOR	5
NAND NAND OR	6
OR OR XOR	7
XOR AND OR	8
XNOR XNOR AND	9

Figure 2.2 The Gate Table I Used for Code Lines (Depends on the Last Digit of my Student Number)

Task 2: Debugging the Design

In this task, I analyzed the provided VHDL project to identify and resolve bugs that prevented successful synthesis and implementation. I found and fixed a total of six errors in the design files.

1. **File Name Mismatch:** The first error was caused by a discrepancy between the file name and the entity name. The file was named sub_module2_the_beast.vhd, but the entity was declared as sub_module2. I renamed the source file to match the entity name to resolve this **(See Figure 2.1.3)**.

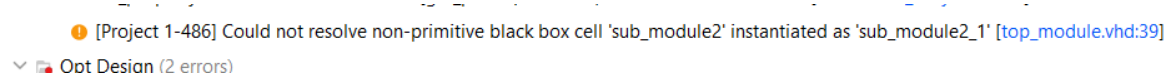


Figure 2.1.1 Error 1

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity sub_module2_the_beast is
5      Port (
6          i_switch_inputs : in  STD_LOGIC_VECTOR (7 downto 0);
7          o_output_vector : out STD_LOGIC_VECTOR (7 downto 0)
8      );
9  end sub_module2_the_beast;
10
11 architecture Structural_sub2 of sub_module2_the_beast is
12     component sub_modu
13         port (
14             i_input_byte  : in  STD_LOGIC_VECTOR (7 downto 0);
15             o_output_byte : out STD_LOGIC_VECTOR (7 downto 0)
16         );
17     end component sub_module1;
```

Figure 2.1.2 Wrong File Name

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity sub_module2 is
5      Port (
6          i_switch_inputs : in  STD_LOGIC_VECTOR (7 downto 0);
7          o_output_vector : out STD_LOGIC_VECTOR (7 downto 0)
8      );
9  end sub_module2;
10
11 architecture Structural_sub2 of sub_module2 is
12     component sub_module1 is
13         port (
14             i_input_byte  : in  STD_LOGIC_VECTOR (7 downto 0);
15             o_output_byte : out STD_LOGIC_VECTOR (7 downto 0)
16         );
17     end component sub_module1;
```

Figure 2.1.3 Corrected File Name

2. **Typo in Signal Name:** A syntax error occurred in the top_module due to a typo in the port map. The signal s_output_1was misspelled as s_ouput_1 (missing the letter 't'). I corrected the spelling to fix the declaration error **(See Figure 2.2.3)**.

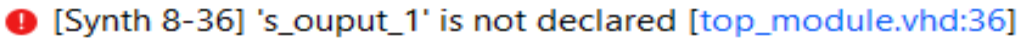


Figure 2.2.1 Error 2

```
30
31  begin
32
33      sub_module1_2 : sub_module1
34          port map (
35              i_input_byte  => i_SW,
36              o_output_byte => s_ouput_1
37          );
38
39      sub_module2_1 : sub_module2
40          port map (s_output_2, i_SW);
41      s_output_3 <= unsigned(s_output_2) + 25;
42
43      o_LED <= (not std_logic_vector(s_output_3)) xor s_output_1;
44
45  end Structural_top;
46
```

Figure 2.2.2 Wrong Name

```
30
31  begin
32
33      sub_module1_2 : sub_module1
34          port map (
35              i_input_byte  => i_SW,
36              o_output_byte => s_output_1
37          );
38
39      sub_module2_1 : sub_module2
40          port map (i_SW, s_output_2);
41      s_output_3 <= unsigned(s_output_2) + 25;
42
43      o_LED <= (not std_logic_vector(s_output_3)) xor s_output_1;
44
45  end Structural_top;
46
47
```

Figure 2.2.3 Corrected Name

3. **Incorrect Port Mapping:** There was a logical connection error in the instantiation of sub_module2. The input signal i_SW and the internal output signal s_output_2 were mapped to the wrong ports. I swapped the connections to ensure the data flowed correctly **(See Figure 2.3.3)**.

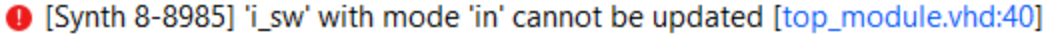


Figure 2.3.1 Error 3

```
30
31  begin
32
33      sub_module1_2 : sub_module1
34          port map (
35              i_input_byte  => i_SW,
36              o_output_byte => s_ouput_1
37          );
38
39      sub_module2_1 : sub_module2
40          port map (s_output_2, i_SW);
41      s_output_3 <= unsigned(s_output_2) + 25;
42
43      o_LED <= (not std_logic_vector(s_output_3)) xor s_output_1;
44
45  end Structural_top;
46
```

Figure 2.3.2 Wrong Ports

```
30
31  begin
32
33      sub_module1_2 : sub_module1
34          port map (
35              i_input_byte  => i_SW,
36              o_output_byte => s_output_1
37          );
38
39      sub_module2_1 : sub_module2
40          port map (i_SW, s_output_2);
41      s_output_3 <= unsigned(s_output_2) + 25;
42
43      o_LED <= (not std_logic_vector(s_output_3)) xor s_output_1;
44
45  end Structural_top;
46
47
```

Figure 2.3.3 Corrected Ports

4. **Syntax Error in Constraints:** The synthesis failed due to a missing character in the constraint file (.xdc). On line 28, a closing curly brace } was missing from the get_ports command. I added the missing brace to fix the syntax **(See Figure 2.4.3)**.

[Designutils 20-970] Unrecognized or unsupported command 'set_property PACKAGE_PIN U19 [get_ports {o_LED[2]]

Figure 2.4.1 Error 4

```
23  # LEDs
24  set_property PACKAGE_PIN U16 [get_ports {o_LED[0]}]
25      set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[0]}]
26  set_property PACKAGE_PIN E19 [get_ports {o_LED[1]}]
27      set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[1]}]
28  set_property PACKAGE_PIN U19 [get_ports {o_LED[2]}]
29      set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[2]}]
30  set_property PACKAGE_PIN V19 [get_ports {o_LED[3]}]
31      set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[3]}]
32  set_property PACKAGE_PIN W18 [get_ports {o_LED[4]}]
33      set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[4]}]
34  set_property PACKAGE_PIN U15 [get_ports {o_LED[5]}]
35      set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[5]}]
36  set_property PACKAGE_PIN U14 [get_ports {o_LED[6]}]
37      set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[6]}]
38  set_property PACKAGE_PIN V14 [get_ports {o_LED[7]}]
39      set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[7]}]
```

Figure 2.4.2 Missing Character

```
# LEDs
set_property PACKAGE_PIN V14 [get_ports {o_LED[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[0]}]
set_property PACKAGE_PIN E19 [get_ports {o_LED[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[1]}]
set_property PACKAGE_PIN U19 [get_ports {o_LED[2]]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[2]}]
set_property PACKAGE_PIN V19 [get_ports {o_LED[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[3]}]
set_property PACKAGE_PIN W18 [get_ports {o_LED[4]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[4]}]
set_property PACKAGE_PIN U15 [get_ports {o_LED[5]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[5]}]
set_property PACKAGE_PIN U14 [get_ports {o_LED[6]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[6]}]
set_property PACKAGE_PIN U16 [get_ports {o_LED[7]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[7]}]
```

Figure 2.4.3 Corrected Character

5. **Wrong Pin Assignment:** A physical mapping error was identified in the constraint file regarding the LEDs. The pin assignments for o_LED[0] and o_LED[7] were swapped/incorrect. I corrected the pin numbers by referring to the Basys3 reference manual **(See Figure 2.5.2)**.

```
# LEDs
set_property PACKAGE_PIN U16 [get_ports {o_LED[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[0]}]
set_property PACKAGE_PIN E19 [get_ports {o_LED[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[1]}]
set_property PACKAGE_PIN U19 [get_ports {o_LED[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[2]}]
set_property PACKAGE_PIN V19 [get_ports {o_LED[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[3]}]
set_property PACKAGE_PIN W18 [get_ports {o_LED[4]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[4]}]
set_property PACKAGE_PIN U15 [get_ports {o_LED[5]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[5]}]
set_property PACKAGE_PIN U14 [get_ports {o_LED[6]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[6]}]
set_property PACKAGE_PIN V14 [get_ports {o_LED[7]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[7]}]
```

Figure 2.5.1 Wrong Pin Assignments

```
# LEDs
set_property PACKAGE_PIN V14 [get_ports {o_LED[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[0]}]
set_property PACKAGE_PIN E19 [get_ports {o_LED[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[1]}]
set_property PACKAGE_PIN U19 [get_ports {o_LED[2]]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[2]}]
set_property PACKAGE_PIN V19 [get_ports {o_LED[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[3]}]
set_property PACKAGE_PIN W18 [get_ports {o_LED[4]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[4]}]
set_property PACKAGE_PIN U15 [get_ports {o_LED[5]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[5]}]
set_property PACKAGE_PIN U14 [get_ports {o_LED[6]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[6]}]
set_property PACKAGE_PIN U16 [get_ports {o_LED[7]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[7]}]
```

Figure 2.5.2 Corrected Pin Assignments

6. **Incorrect Target Device:** Finally, the project was initially configured for the wrong FPGA part. I updated the project settings to target the **xc7a35tcpg236-1**, which is the correct Artix-7 chip used on the Basys3 board **(See Figure 2.6.3)**.

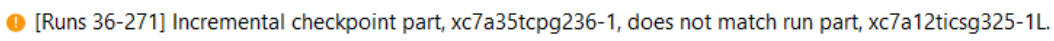


Figure 2.6.1 Error 6

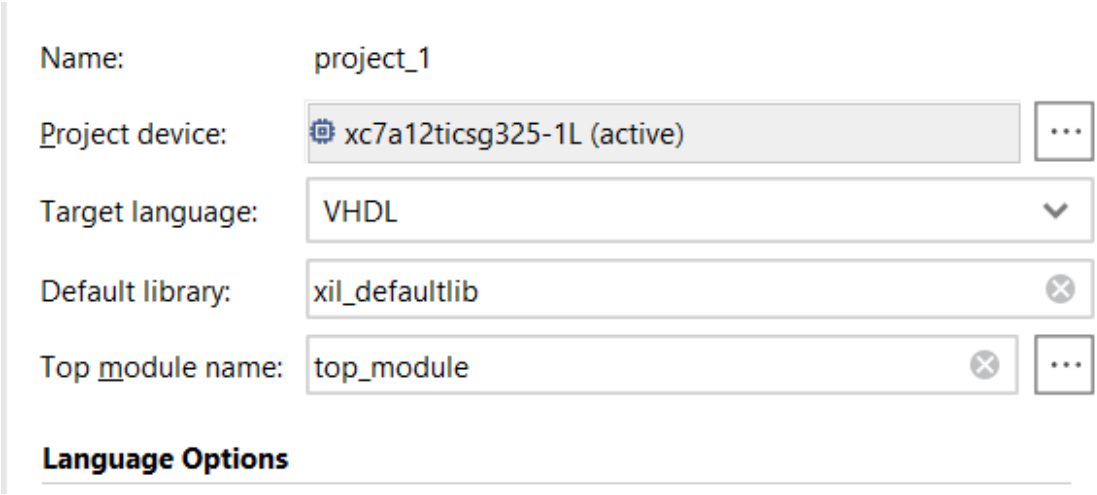


Figure 2.6.2 Wrong Project

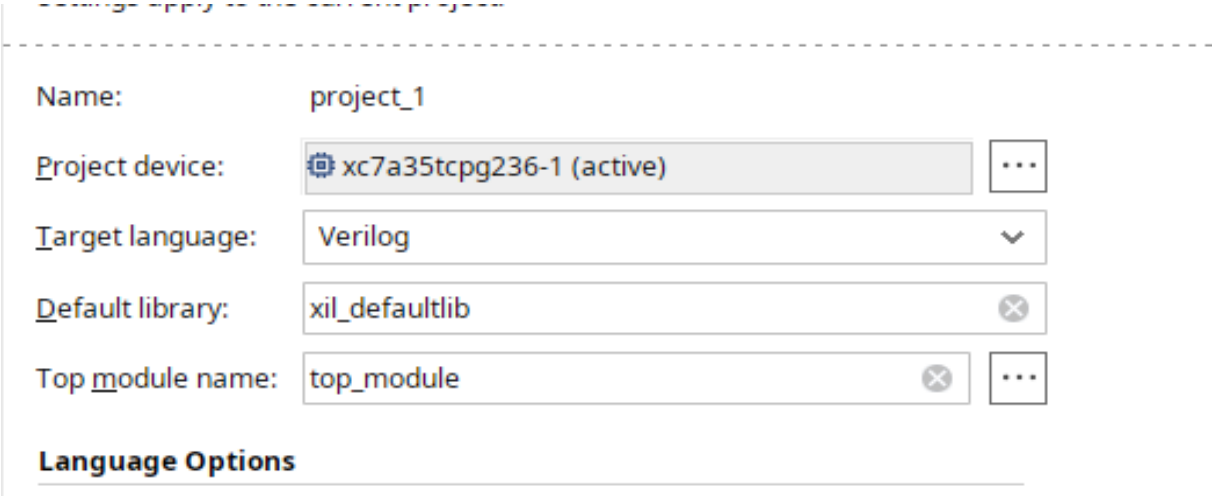


Figure 2.6.3 Corrected Project

Task 3: FPGA Implementation

Following the debugging process, I successfully synthesized and implemented the corrected VHDL design on the Basys3 FPGA board. The synthesis and implementation runs completed without errors, and the bitstream file was generated.

Using the Vivado Hardware Manager, I programmed the device and verified the circuit's operation in real-time. The on-board switches were used as inputs (i_SW), and the LEDs (o_LED) displayed the corresponding outputs. I tested various input

combinations to ensure that the logic gates (OR, OR, XOR) and the arithmetic operations functioned as expected. The physical verification of the design is demonstrated in **Figure 3.1** through **Figure 3.7**, which show the board responding to different switch configurations.

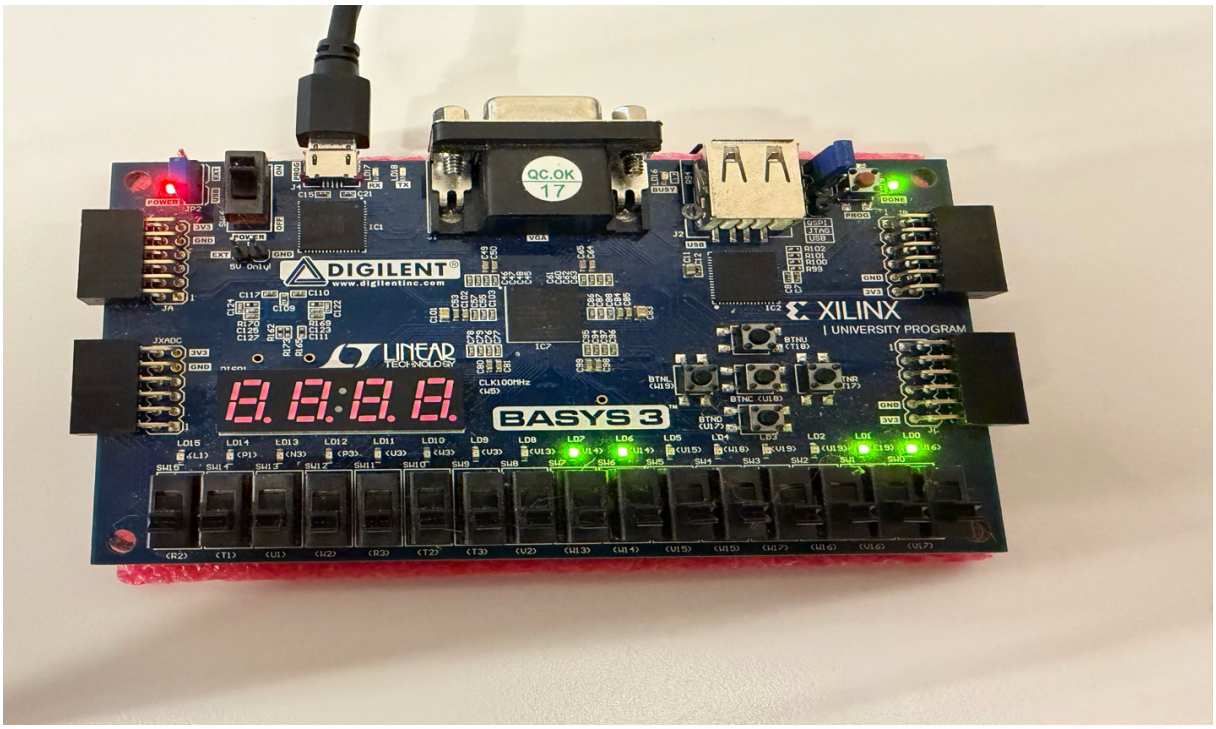


Figure 3.1 Input: 00000000 Output: 11000011

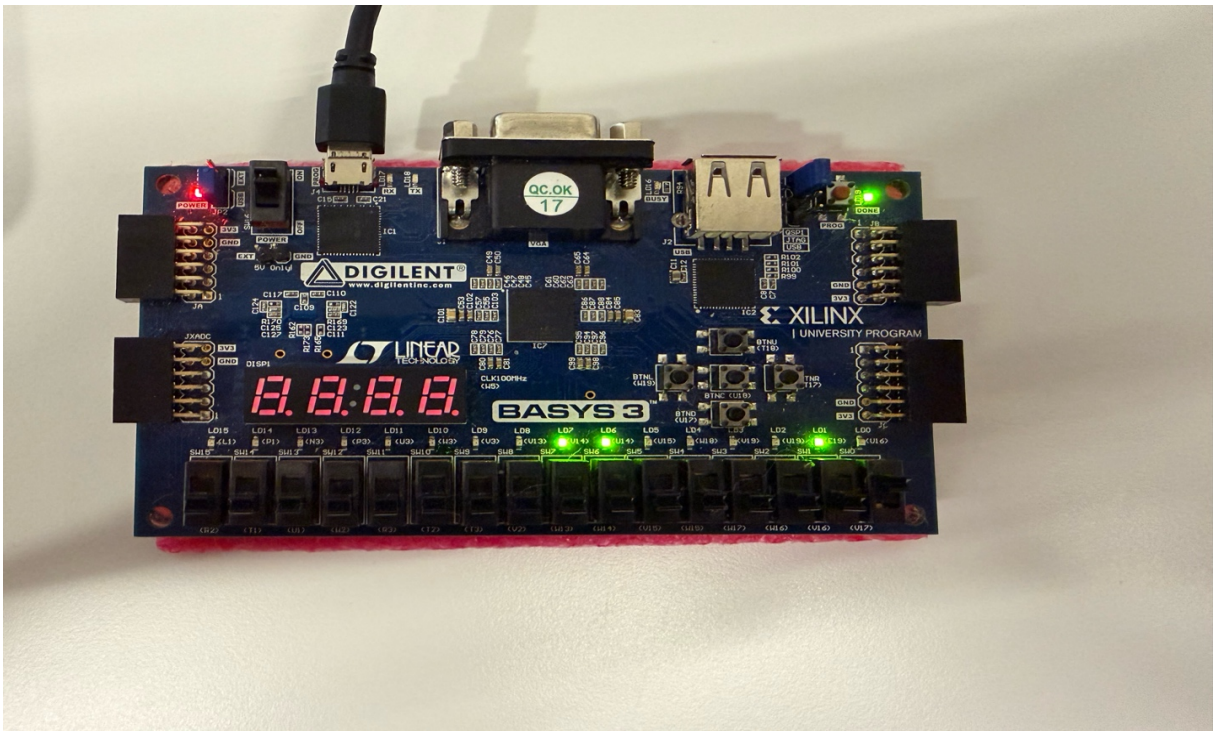


Figure 3.2 Input: 00000001 Output: 11000010

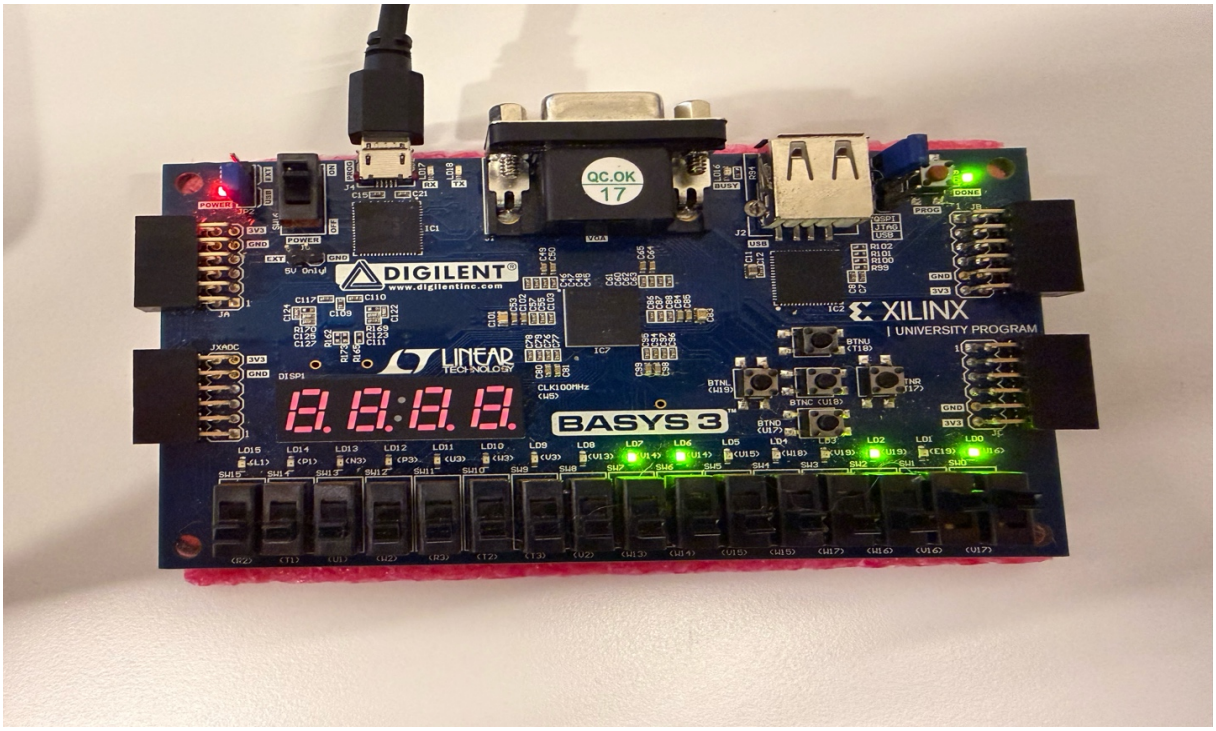


Figure 3.3 Input: 00000011 Output: 11000101

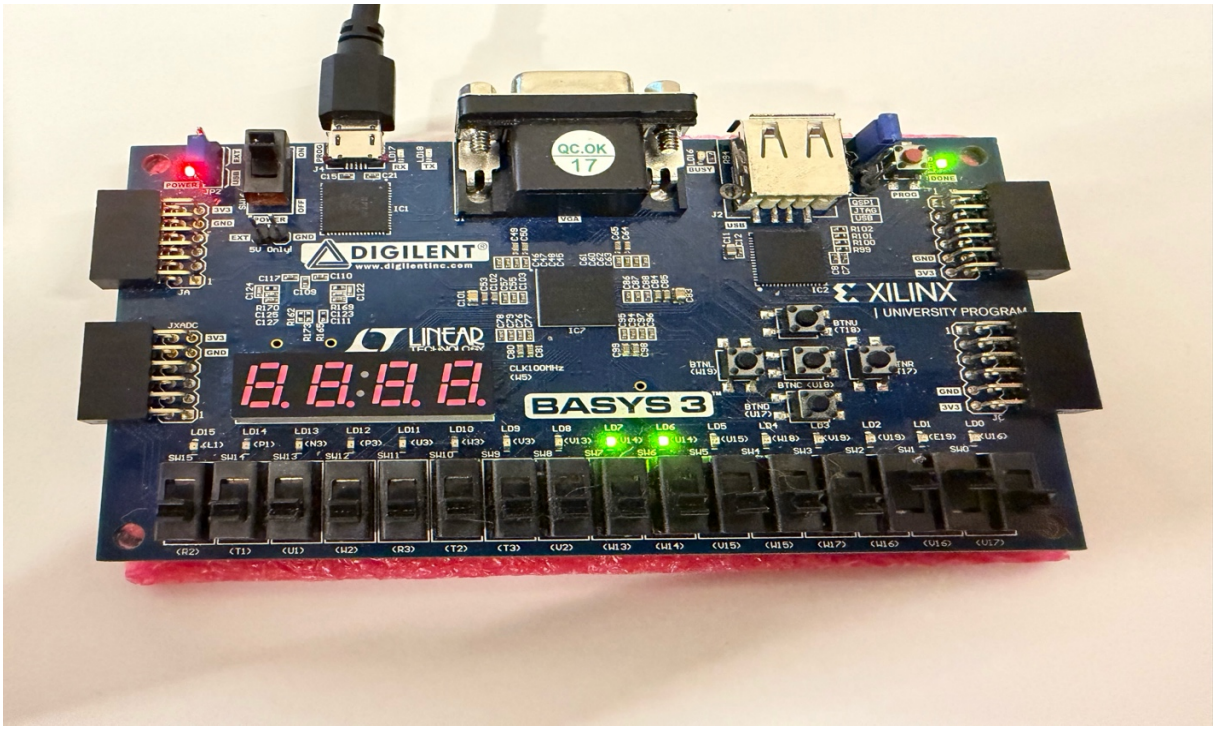


Figure 3.4 Input: 00000110 Output: 11000000

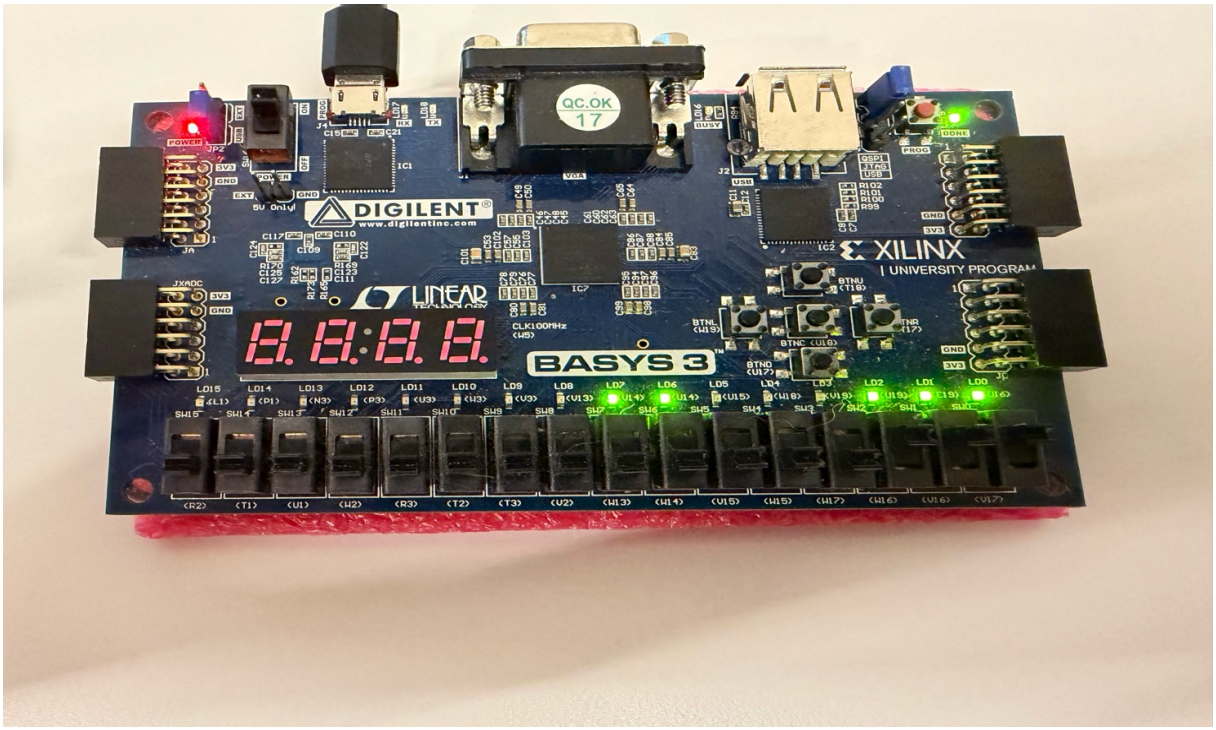


Figure 3.5 Input: 00000111 Output: 11000111

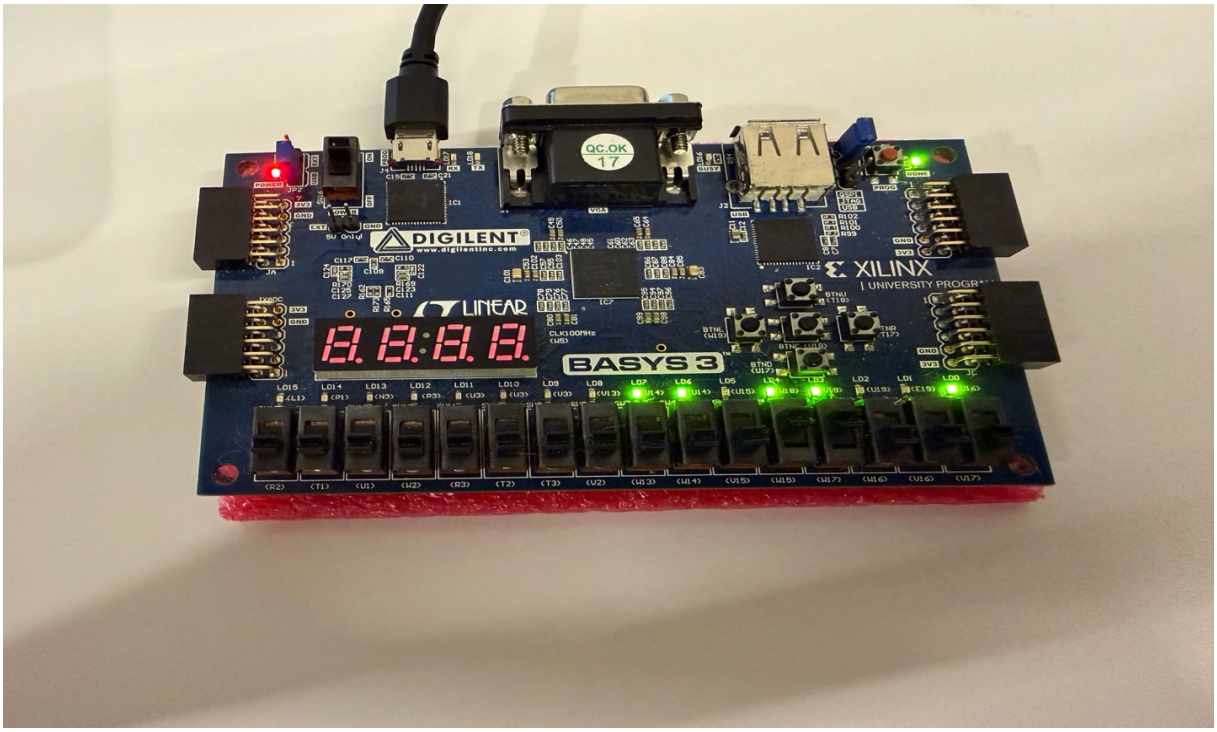


Figure 3.6 Input: 00011000 Output: 11011001

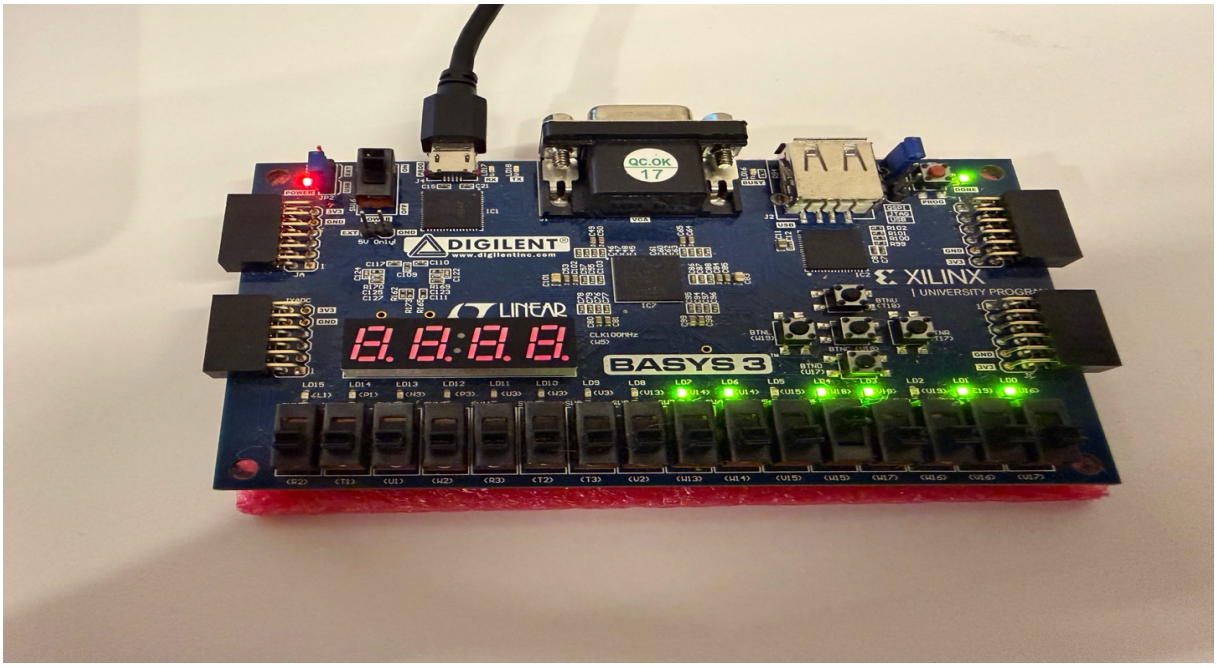


Figure 3.7 Input: 00010000 Output: 11011011

Task 4: Testbench Simulation

To verify the functional correctness of the design before hardware implementation, I developed a VHDL testbench. The simulation was structured to exhaustively test the circuit by iterating through all 256 possible binary combinations of the 8-bit input vector (i_SW).

In the simulation code, a delay of **10 ns** was applied between each input change to allow for stable signal propagation. The resulting waveform in Vivado confirmed that the circuit outputs (o_LED) change correctly in response to every input state. These simulation results were consistent with the physical behavior observed on the Basys3 board, validating the logical design.

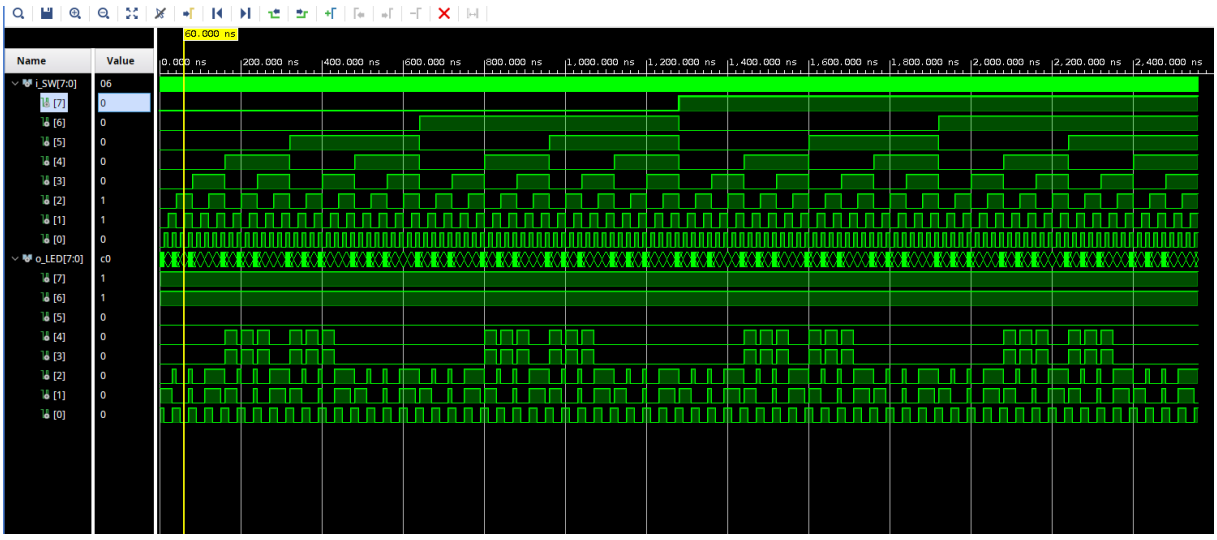


Figure 4 Simulation Results for 256 Combinations

Task 5: Schematic Analysis

In the final phase of the experiment, I generated and analyzed the circuit's structure using three different schematic views available in Vivado: the RTL analysis, the Synthesized design, and the Implemented design.

RTL Schematic: This view provided a high-level representation of the VHDL code, clearly displaying the hierarchy between the top_module and its sub-modules, along with the signal flow between inputs and outputs.

Synthesized Schematic: This schematic showed how the VHDL code was translated into actual logic elements, such as Look-Up Tables (LUTs) and buffers, revealing the optimized logic structure before physical placement.

Implemented Schematic: Finally, this view demonstrated the mapping of the synthesized logic onto the specific physical resources of the Artix-7 FPGA.

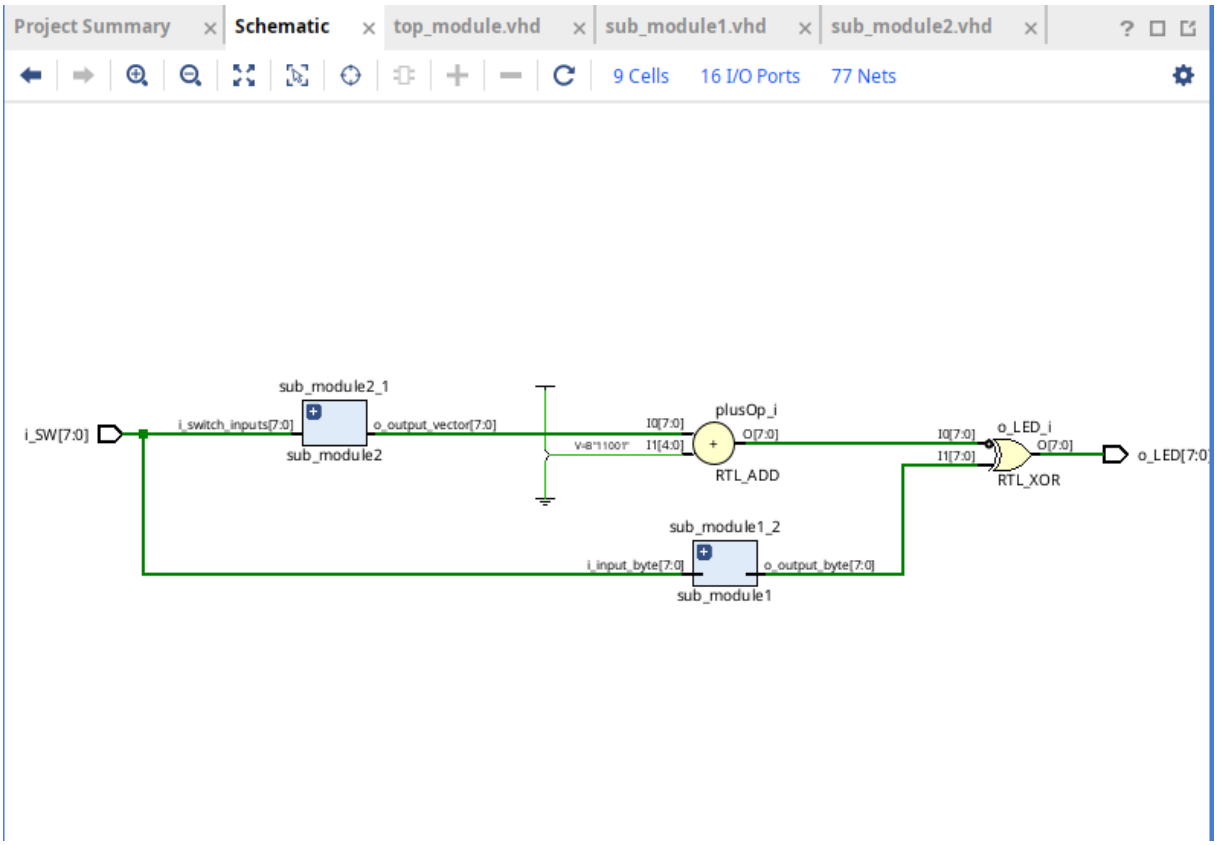


Figure 5.1 RTL Schematic

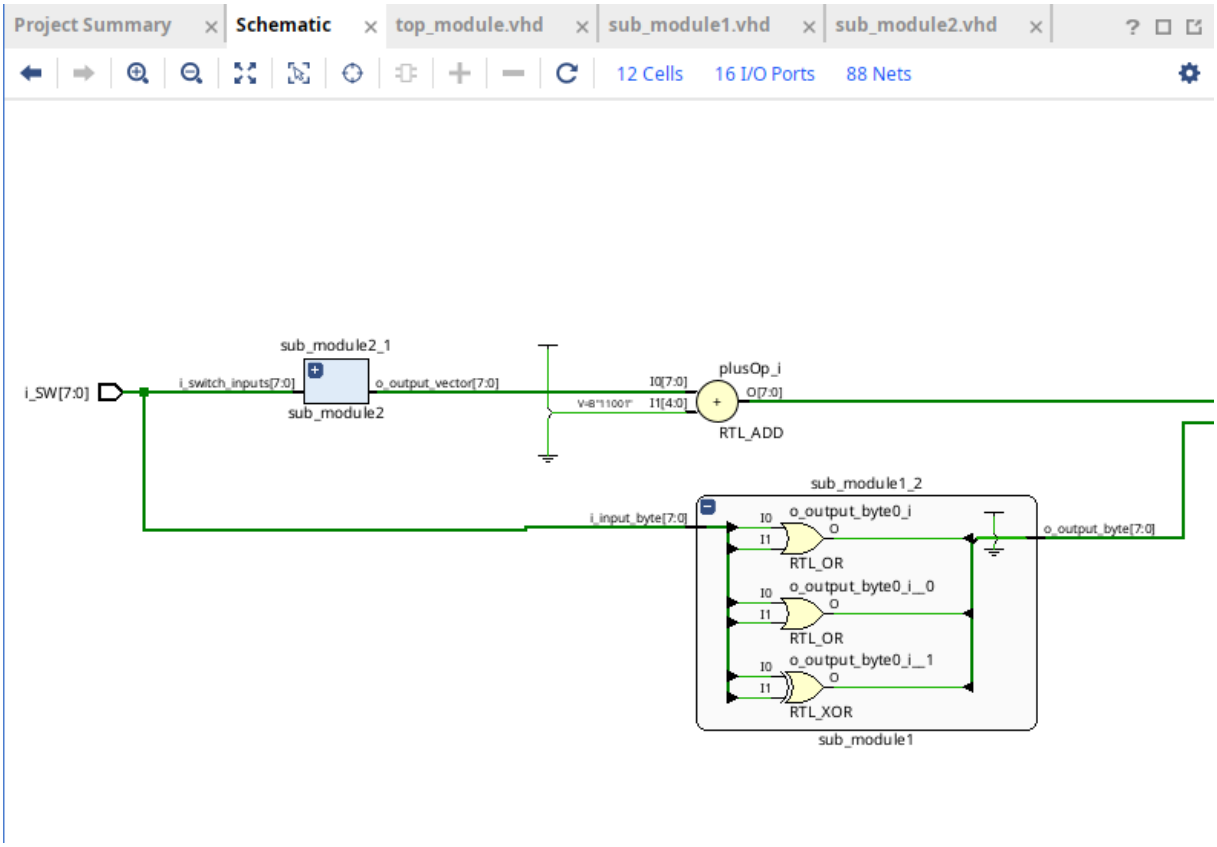


Figure 5.2 RTL Schematic 2

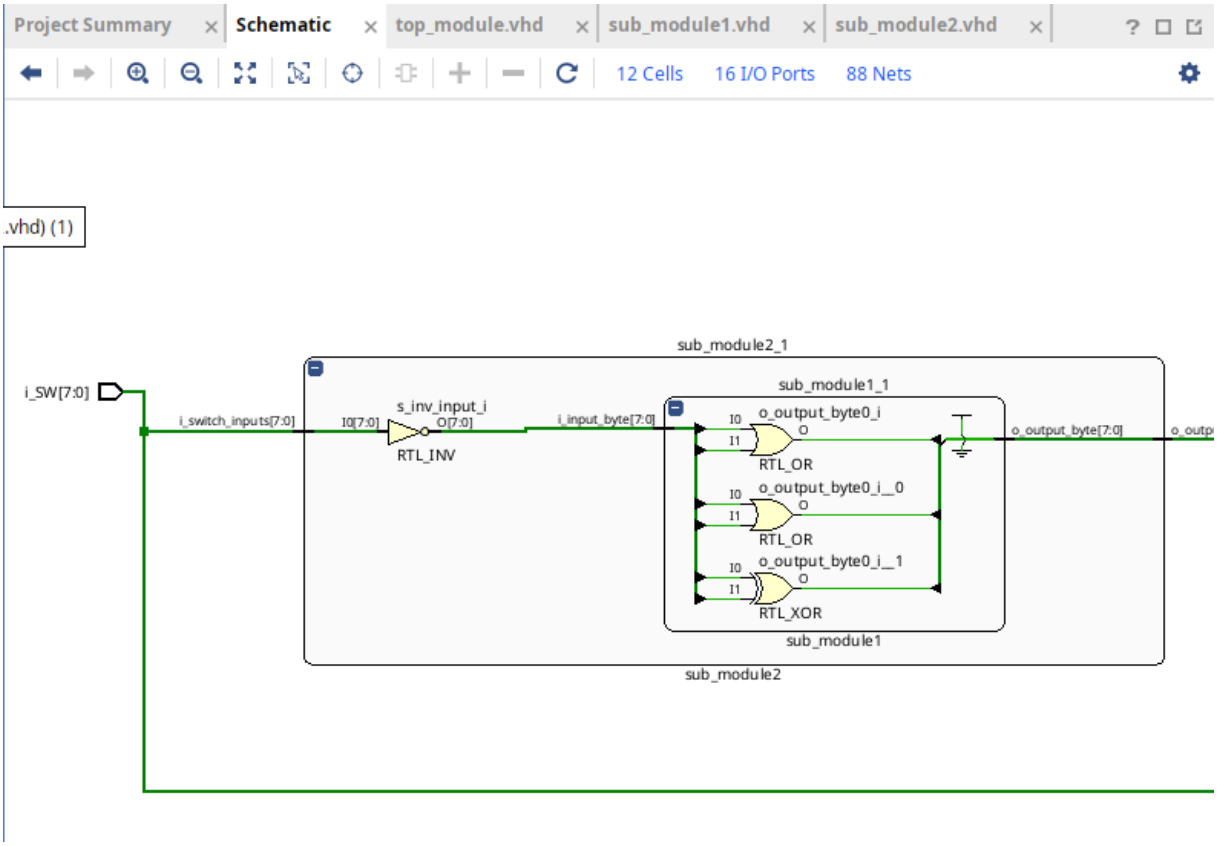


Figure 5.3 RTL Schematic 3

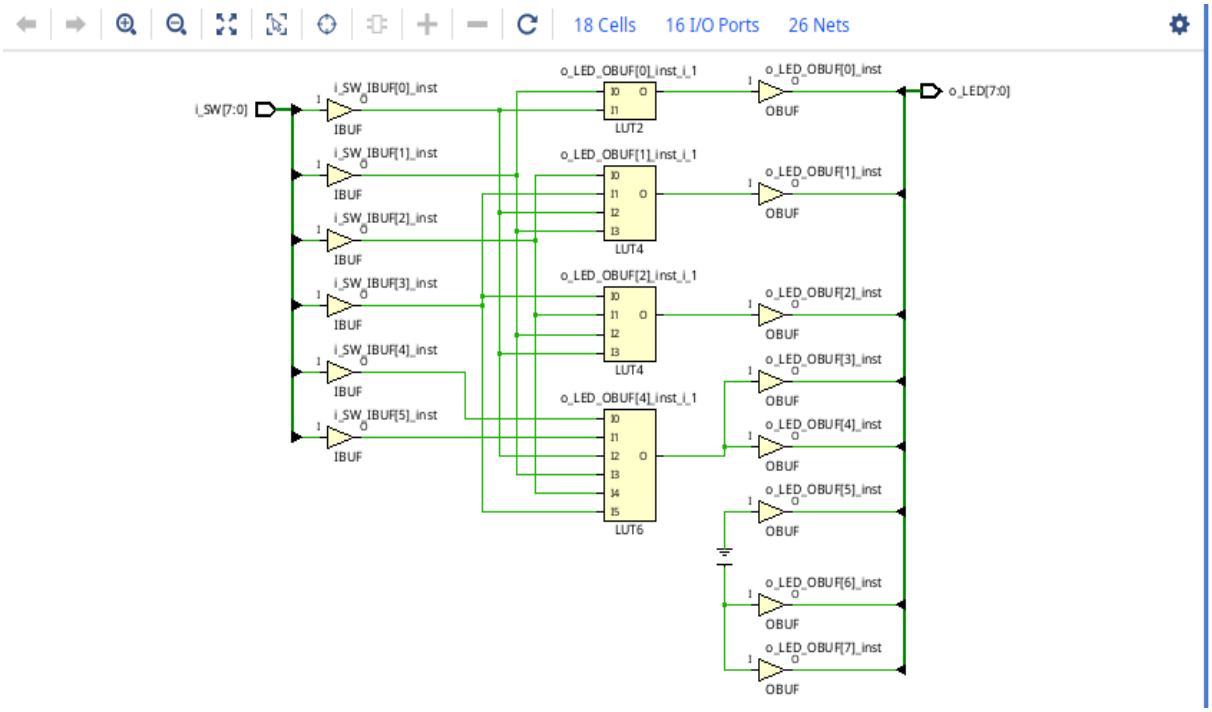


Figure 5.4 Synthesized Schematic

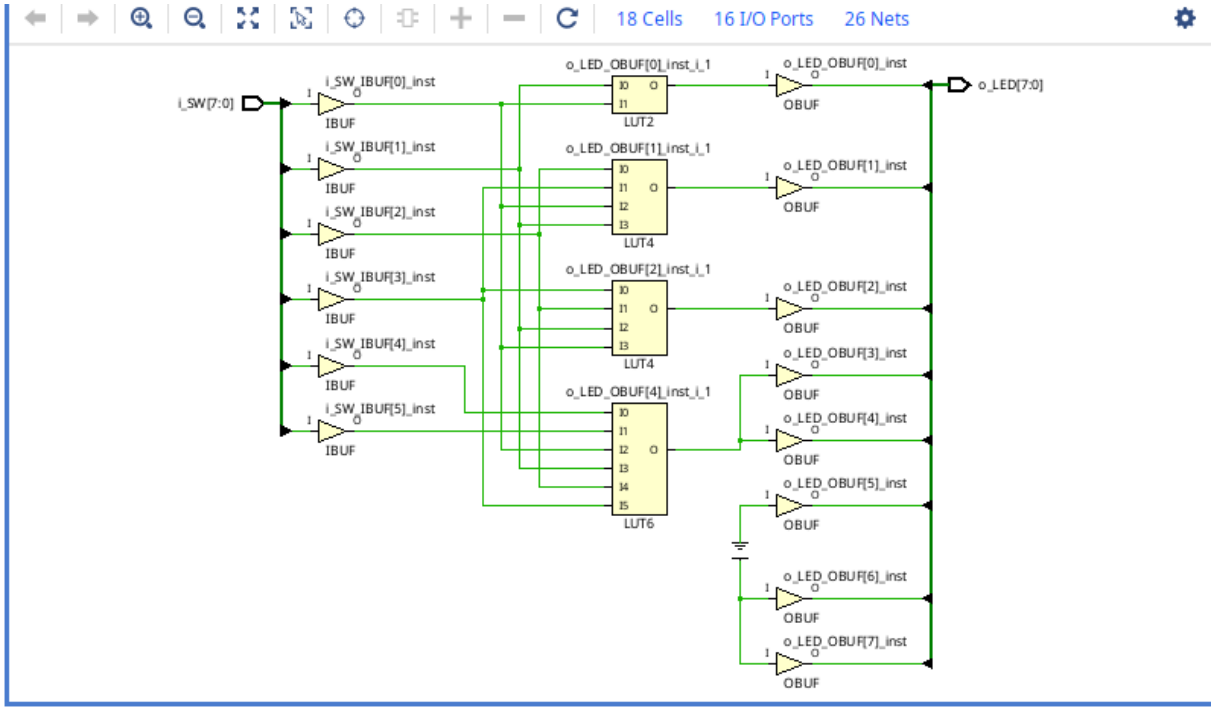


Figure 5.5 Implemented Design

Conclusion

In this experiment, I successfully completed the full design cycle of a combinational logic circuit, starting from VHDL coding to physical implementation on the Basys3 FPGA. The process of debugging the provided code was particularly valuable, as it reinforced my understanding of VHDL syntax, hierarchical port mapping,

and the critical role of constraint files in assigning hardware pins. By writing a testbench and simulating the design, I verified the logic before deployment, ensuring that the circuit functioned correctly under all input conditions.

The final hardware implementation demonstrated that the LEDs responded accurately to the switch inputs, confirming that the modified logic gates (OR, OR, XOR) and arithmetic operations were executed as intended. Furthermore, analyzing the RTL, Synthesized, and Implemented schematics provided a clear visualization of how high-level code is translated into physical logic resources. Overall, this laboratory session established a solid foundation in using the Vivado Design Suite and understanding the practical relationship between HDL code and physical hardware behavior.

Appendices

Test Bench Code:

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity tb_top_module is

end tb_top_module;

architecture Behavioral of tb_top_module is

  -- Signals to connect to the DUT

  signal i_SW  : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');

  signal o_LED : STD_LOGIC_VECTOR(7 downto 0);

  -- Component declaration of the top module

  component top_module is

    Port (

      i_SW  : in  STD_LOGIC_VECTOR (7 downto 0);

      o_LED : out STD_LOGIC_VECTOR (7 downto 0)

    );

  end component;
```

```
begin
  -- Instantiate the Device Under Test (DUT)
  DUT : top_module
    port map (
      i_SW  => i_SW,
      o_LED => o_LED
    );
  -- Stimulus process: apply all 256 combinations sequentially
  stim_proc : process
  begin
    for i in 0 to 255 loop
      i_SW <= std_logic_vector(to_unsigned(i, 8));
      wait for 10 ns;
    end loop;
    wait; -- stop simulation
  end process;
end Behavioral;
```

References

- BASYS 3 – Vivado Tutorial.pdf
- Basys3xdc.pdf
- Test_Bench_Tutorial.pdf
- Nandland, "Introduction to VHDL for Beginners with Code Examples," [Online]. Available: https://nandland.com/introduction-to-vhdl-for-beginners-with-code-examples/
- VHDLwhiz, "VHDL Port Map," [Online]. Available: https://vhdlwhiz.com/port-map/
- Digilent Inc., "Basys 3 FPGA Board Reference Manual," [Online]. Available: https://digilent.com/reference/programmable-logic/basys-3/start
- AMD Xilinx, "Vivado Design Suite User Guide," [Online]. Available: https://www.xilinx.com/support/documentation-navigation/design-hubs/dh0012-vivado-getting-started-hub.html
- ASIC World, "VHDL Tutorial," [Online]. Available: http://www.asic-world.com/vhdl/index.html